SOTER: Guarding Black-box Inference Service for Neural Networks at the Edge

***Abstract*—Tremendous AI applications empowered by deep learning inference are increasingly deployed on edge devices to provide an interactive user experience. This necessitates securely offloading the service provider's proprietary model from the cloud to the edge, and using a co-located accelerator (e.g., GPU) to speed up model inference locally. A pragmatic approach is to use Trusted Execution Environments (TEEs) to protect the offloaded model from the untrusted software stack. However, despite much effort, securely outsourcing inference computations from a TEE to a faster, yet untrusted GPU remains a fundamental challenge. Existing TEE-based solutions either ban the choice of GPU acceleration or have to retrain the model to partition insensitive model layers to run in a GPU, requiring substantial expertise spanning AI algorithms and causing an accuracy drop.**

We propose SOTER, a secure inference system that enables outsourcing sensitive inference computations from a TEE to a co-located untrusted GPU automatically and has zero accuracy drop. Our key idea stems from the observation that most heavyweight model operators have an *associativity* property. With *associativity*, SOTER automatically profiles and partitions operators with morphed parameters to a GPU. SOTER introduces hidden execution of these morphed operators in GPU and recovers the real execution results in TEE when needed. By adopting the *morph then recover* technique, SOTER avoids tuning the original model parameters and thus causes zero accuracy drop; In addition, SOTER designs an *oblivious fingerprinting* technique to safely detect integrity breaches of outsourced operators. Based on an extensive evaluation of nine state-of-the-art deep learning models, SOTER demonstrates an encouraging latency reduction by up to 19.1× over baseline systems on complex big models NASNet and PNASNet while maintaining high inference accuracy. Meanwhile, evaluation results on model integrity attacks show that SOTER can quickly detect integrity breaches with high probability and zero false positives.

I. INTRODUCTION

Driven by the remarkable success of Deep Learning (DL) [12], [41], edge computing applications are trending to utilize Deep Neural Networks (DNNs) to help analyze real-time data by taking inputs (e.g., images) and transforming those inputs into actionable insights (i.e., inference). For instance, autopilot navigation [4], [7], industrial automation [18], speech recognition [28] and visual assistance [33] are pervasive examples powered by DNN models (in short, models). Traditionally, AI service providers host their proprietary models on the cloud and provide inference service interfaces to clients. To utilize the model, clients can send inference queries to the cloud and receive results through a web portal [41].

While traditional inference services are primarily hosted on the cloud, increasingly inference workloads are run natively on edge devices for two reasons. First, numerous edge computing applications require millisecond level responses to conduct real-time data analysis [16], [40], which makes a cloud-based approach unsuitable because clients have to transfer high volume of queries over the network [31]. Second, since clients' queries can be sensitive (e.g., images for face recognition), existing cloud-based approach leverages homomorphic encryption [11], [15] and Garbled Circuits [2] to protect clients' data, coming at substantial computational overhead. For example, in Delphi [31], a state-of-the-art cloud-based secure inference system, making one inference query requires ∼60MB communication and ∼4s latency, violating the stringent latency requirement of real-time applications.

In this paper, we summarize three crucial requirements for offloading inference services to the edge. First (**R1**), the offloaded model should be kept secret with *black-box* access. *Black-box* means that the client can query the model as an oracle but will learn nothing more than the outputs (e.g., class labels for images). This requirement is critical to protect service providers' proprietary model that has been trained with substantial resource and efforts because publishing models with *white-box* accesses exposes a large attack surface to the adversary [39], [29]. Second (**R2**), the inference computations should be fastly processed with a co-located accelerator (e.g., GPU) in order to meet the stringent latency requirement of running applications. Third (**R3**), the offloaded model should achieve the same inference accuracy as the original model on the cloud without any accuracy drops.

To ensure the offloaded model can be queried as a *black-box* oracle (**R1**), a recent pragmatic approach is to leverage Intel SGX [6], a pervasively used Trusted Execution Environment (TEE) to host models. SGX provides code integrity and data confidentiality for enclaves, such that the offloaded model hosted in enclave cannot be seen or tampered with from other untrusted software stack running on the client's platform. Moreover, by leveraging the attestation tool of SGX, service providers can perform remote attestation to the client and ensure that the correct model is safely running at the client device equipped with SGX.

Unfortunately, SGX is not a panacea to solve the problem. Although many existing SGX-based systems can protect the offloaded models with secure enclaves, none of them can meet these three requirements simultaneously. As a first step, Hanzlik proposes MLcapsule [19], a modular approach that hosts the entire model layers in enclaves to guarantee model confidentiality (meeting **R1**). However, MLcapsule runs model inference purely with CPU and bans the choice of GPU ac-

Network and Distributed Systems Security (NDSS) Symposium 2021
21-24 February 2021
ISBN 1-891562-66-5
https://dx.doi.org/10.14722/ndss.2021.23xxx
www.ndss-symposium.org

celeration, achieving unsatisfactory latency results for diverse inference workloads, failing to meet **R2**.

We believe a fundamental challenge lies in the confidentiality dilemma between *black-box* access (**R1**) and fast execution with a co-located accelerator (**R2**): either the client has to enclose all inference computations within enclave to guarantee model confidentiality, or outsource part of sensitive models to the untrusted accelerator for faster processing.

Confronted with the confidentiality dilemma, known remedies including Alexander et al. [34] and Gu et al. [17] take a partition approach to leverage the strong computation power of a co-located GPU. Specifically, in these approaches, the client has to retrain the original model with transfer learning techniques [35] and then split the retrained model into public and private layers. By holding the private layers in enclave and outsourcing public layers to GPU, sensitive parameters in the private layers remain confidential while the untrusted hardware accelerator can handle the public layers fastly, achieving lower inference latency than pure CPU inference with SGX (i.e., MLcapsule).

However, although existing partition-based solutions can speedup inference computations on offloaded models, they face a paradox between low-latency inference (**R2**) and accurate inference (**R3**). By default, these approaches require the client to manually decide a proper layer partition plan. If the client partitions more layers to GPU for faster inference, the client has to re-train more sensitive parameters, incurring prohibitive accuracy drops (up to 6% in our evaluation). If the client partitions less layers to GPU, the inference would get a more accurate result because less sensitive parameters of the original model are tuned, but such partition brings significant latency explosion because more inference computations are run in CPU. Moreover, as DNN models are becoming bigger and increasingly complex with the rapid advancement of neural architecture search (NAS) [5], [10], it is hard to manually decide an optimal partition plan because a big model might consist numerous cycles between layers, a straight-forward manual partition could incur frequent kernel switches between CPU and GPU, significantly cutting down the performance gain by outsourcing computations to GPU.

To this end, we propose SOTER, the first system that provides black-box inference services at the edge while meeting all three crucial requirements. Without compromising model confidentiality, SOTER can automatically partition a fraction of intensive inference computations to the untrusted GPU without retraining the original model, thus incurring zero accuracy drops.

Our key weapon to meet all three requirements simultaneously is the *associativity* property of DNN operators. *Associativity* means that the way that factors grouped in an operator's computation does not change the final result. Since most computational-intensive DNN operators are associative (e.g., *Convolution*, *Linear*), SOTER carries a *Morph Then Recover* (MTR) protocol to securely partition heavyweight inference computations to GPU without compromising confidentiality. Specifically, before the inference, SOTER profiles the entire model to learn the architecture and *associative* operators to find an optimal partition plan. Then, SOTER morphs the parameters of a fraction of *associative* operators with randomly generated *blinding coins* (unsigned integers) in enclave, safely partitions these *morphed* operators to GPU for faster inference computing, and then *recovers* the real inference result with reciprocal coins in enclave when needed.

Unfortunately, even with the MTR protocol, it is still challenging to ensure accurate model inference (**R3**) because the untrusted accelerator might not perform inference correctly. Although the parameters of partitioned operators have been morphed and remain confidential, the adversary can still modify partitioned operators' parameters or alter the intermediate inference results, misleading an offloaded model to produce wrong outputs.

SOTER addresses this challenge with an *oblivious fingerprinting* technique. *Fingerprints* are a set of transformed inputs and their corresponding model outputs, which hold the ground-truth correctness of model operators. SOTER creates a set of *fingerprints* for each partitioned operator in a *pre-processing* phase where all inference computations are run in enclave to ensure no perturbation occurs. Then, in an *inference* phase, SOTER verifies the integrity of outsourced operators by injecting checking queries (i.e., *fingerprint* inputs) into client's normal queries, and comparing the computed outputs with the expected operators' outputs (i.e., *fingerprint* outputs) in enclave. Any mismatches indicate integrity breaches of partitioned operators outside enclave.

However, using only a static set of *fingerprints* to detect integrity breaches is not secure since the adversary can observe the fixed *fingerprint* inputs through long-term monitoring and evade the integrity checking. SOTER tackles this problem by making *fingerprints* oblivious. Specifically, with a fixed set of *fingerprints* after pre-processing, SOTER dynamically derives more oblivious *fingerprints* by randomly assembling several existing *fingerprints* with random coefficients in enclave. The newly generated *fingerprints'* inputs statistically share the same distribution as the normal inputs (proved in section §IV) thus the adversary outside enclave can not distinguish the checking queries (i.e., *fingerprint* inputs) and evade our verification.

We implemented SOTER on the Slalom [38] codebase, a mature and lightweight C++ framework for writing neural networks. We evaluated SOTER with nine state-of-the-art DNN models covering four mainstream model types: Multi-layer Perception (MLP), Convolution Neural Network (CNN), Recurrent Neural Network (RNN) and Transformer. We compared SOTER to two state-of-the-art TEE-based secure inference systems (MLcapsule [19] and eNNclave [34]). Our evaluation shows that:

- **R1:** SOTER provided strict black-box access to offloaded models and SOTER can quickly detect integrity breaches of partitioned operators outside enclave with high accuracy and zero false positives (§IV-D).
- **R2:** SOTER was efficient. SOTER's inference latency was 3.5×∼ 22.6× lower than the pure TEE inference baseline (MLcapsule), and has up to 1.8× latency reduction over the partition-based baseline (eNNclave) (§V-A).
- **R3:** SOTER was accurate. SOTER can achieve the ground-truth accuracy of the original model while existing partition-based baseline (ennclave) could cause 1.1%∼6.3% accuracy drops (§V-B).

Our major contribution is SOTER, a new black-box inference system that enforces all the three requirements for secure model inference at the edge. Compared to existing TEE-based secure inference systems, SOTER takes the first step to meet all the three requirements, making SOTER unique to support mission-critical AI applications at the edge (e.g., Home monitoring [9], [37], visual assistance [33]). Besides, SOTER's *oblivious fingerprinting* technique can be integrated to other TEE-based inference systems to help them detect integrity breaches outside the TEE.

In the remaining of this paper, §II introduces background. §III gives an overview of SOTER. §IV describes SOTER's design and security analysis. §V shows our evaluation. §VII discusses related work and §VIII concludes.

II. BACKGROUND

A. Deep Neural Network

With the rapid advancement of deep learning, AI models powered by deep neural networks (DNNs) are increasingly deployed at the edge to serve mission-critical applications []. A Deep Neural Network (DNN) can be represented as a sequence of connected *layers* with each layer assigned a set of *operators*. A DNN operator is either a *linear operator* or a *nonlinear operator* where a linear operator is weighted by a set of pre-trained *parameter matrices* (in short, parameters).

The life-cycle of a DNN model can be divided into two phases: a training phase and an inference phase. In the training phase, to obtain an accurate model, the model provider designs an optimal model architecture, passes the model with his proprietary training dataset multiple times, and updates the numerical values of parameters by calculating the gradients with forward-backward propagations []. Both the model architecture and parameters are sensitive to a third-party as they require substantial training efforts and expertise knowledge []. In the inference phase, all parameters are fixed and do not need to be updated. Without losing generality, we use image classification as an example to illustrate the DNN model in next discussions (shown in Figure 1).

Inference workflow. Figure 1 shows the workflow of running an inference on a pre-trained DNN with well-designed architecture and parameters. A DNN model M takes an input X (e.g., an image) and maps it to a corresponding output Y (e.g., a class label), which can be defined as a function $M : X \Rightarrow Y$. Specifically, an inference phase takes three steps. First, the model M passes the input X through all the layers and computes $Z(X)$ which is known as the *logits* []. Second, M utilizes the *softmax* function to normalize the values of $Z(X)$ and computes an output vector $V_i(X) = softmax(Z(X))$, where each element in $V_i(X)$ represents the probability of the input X being class i while holding $V_i(X) \in (0, 1)$ and $\sum V_i(X) = 1$. Last, M computes and assigns a class label $Y = argmax_i V_i(X)$ to the input X.

DNN operators. Operators are the key building blocks of a DNN where linear operators are the main computational bottleneck in inferences. For instance, for VGG16 inference on a single CPU thread, about 98% of the computation is spent on the linear operators []. Linear operators have two typical forms: *convolution* and *fully-connected*. During the convolution

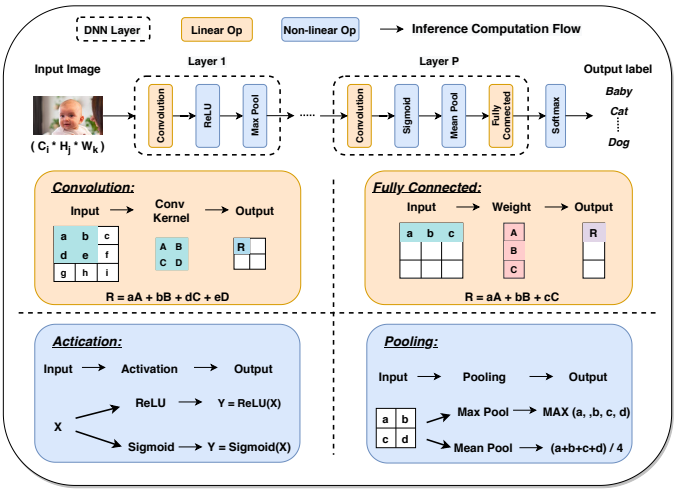


Fig. 1: An overview of a DNN model.

computation, the convolution kernel (with its parameters) is placed at different locations of the input data. At each location, a sum of element-wise product is computed between the kernel and corresponding input values within the kernel window. The fully-connected operator conducts a weighted sum (i.e., a dot product) between an input and a flatten parameter vector which is owned by the fully-connected operator.

Nonlinear operators typically use *activation* functions, such as *Rectified Linear Unit (ReLU)*: $f(x) = max\{0, x\}$, and *Sigmoid*: $f(x) = 1/(1 + e^{-x})$ to approximate complex transformations [], and use *pooling* functions, such as the *max-pool* and *mean-pool* to reduce the dimension of intermediate computation results produced in inference computations.

Associativity. Suppose we have an input data X and a non-zero number μ, a DNN operator F is associative if

$$(\mu^{-1} * \mu)F(X) = \mu^{-1}F(\mu X) \quad (1)$$

Typical linear operators (e.g., convolution, fully-connected) all have the associativity property because these operators conduct linear transformations on the input data. Take the most commonly used, computational-expensive convolution operator [] as an example. As shown in Figure 1, if we multiply each element in the convolution kernel by 2^{-1} (i.e., $\mu = 2^{-1}$), we will get the output $2^{-1}R$ while it always holds $(2 * 2^{-1}) * R = 2 * (2^{-1}R)$. Hence, the convolution operator is associative. This is applicable to the fully-connected operator as well.

Since nonlinear operators conduct nonlinear transformations on the input data, nonlinear operators usually do not have an associativity property (e.g., Sigmoid). However, we observe that under specific constraints, some nonlinear operators can also be associative. Specifically, when μ>0, the ReLU operator is associative because μ will not change the positive or negative of the input X, such that $\mu^{-1}max\{0, \mu X\}$ will get the correct output. The same constraint is applicable to the max-pool operator as well. Note that the mean-pool operator naturally have the associativity property because it actually conducts linear transformation (i.e., averaging) on an input data.

Operators that satisfy Equation 1 also meets a variant of

the associativity property. Given that

$$\begin{cases} F(X_1) = Y_1, \\ F(X_2) = Y_2. \end{cases}$$

we have

$$F(\mu_1 X_1 + \mu_2 X_2) = \mu_1 Y_1 + \mu_2 Y_2 \quad (2)$$

SOTER leverages both Equation (1) and Equation (2) as the key weapon to meet the three crucial requirements (**R1**∼**R3**) simultaneously. We illustrate our detailed designs in §IV.

B. Intel SGX

Intel Software Guard eXtension (SGX) [6] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called an enclave, where data and code execution cannot be seen or tampered with from outside. Code outside enclaves can enter an enclave by `ECalls`, and SGX uses remote attestations [6] to prove that a particular piece of code is running in an enclave on a genuine SGX-enabled CPU. SGX provides a trustworthy random source (`sgx_read_rand`), which invokes a pseudo-random generator through the RDRAND CPU instruction seeded by on-chip entropy sources [6]. Previous studies show that this random source complies with security and cryptographic standards and cannot be seen or tampered with from outside enclaves [3], [32].

MLcapsule [19] encapsulates individual model layers and run inference computations purely inside the SGX. Such a modular approach makes it east to combine layers and form large networks but fails to provide fast inference speed (**R2**) due to the computational bottleneck of CPU.

Same as SOTER, eNNclave [34] and DeepEnclave [17] also take a partition-based approach to utilize a co-located GPU for fast processing. However, these two systems require manual effort to partition and re-train part of the model to make the parameters of the re-trained part insensitive, and then outsource the insensitive part to GPU. This necessitates finding a performance-accuracy trade-off and fails to meet **R3**.

III. OVERVIEW

A. System setup

We consider a local inference scenario shown in Figure 2. There are two parties in the system setup: the client and the model provider (or server). The client owns private inference data for a specific AI task. The server owns an appropriate model that has well-trained model architecture and parameters for the task. The server offloads its model to an SGX-equipped edge device. The offloaded model conducts local inference by taking client's sensitive query, then sends the inference result back to the client.

B. Threat model and privacy goals

Similar to other SGX-based secure inference systems, namely the MLcapsule [19], eNNclave [34] and DeepEnclave [17], SOTER is designed for a two-party setting, where both parties have a bounded computational capability as in [43], [45], and try to learn information about the other parties' private data from the messages it receives. That is,

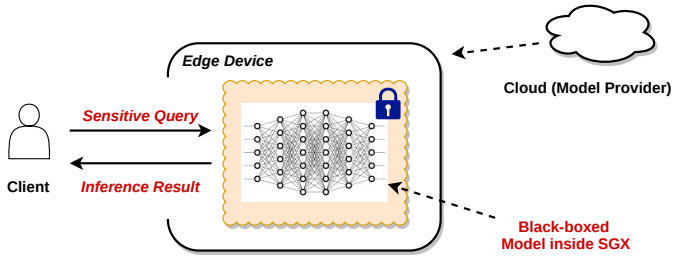


Fig. 2: The local inference scenario handled by SOTER.

the client C and server S will follow the inference protocol coded inside SGX, but the client C wants to learn the model architecture and parameters and the server S attempts to learn the client's input data. Note that, different from prior work that assumes the offloaded model is built on well-known DNN architectures such as ResNet [] and VGG-16/19 [], we intend to protect both the parameters and model architecture (number of layers, linkages between layers, etc) because recent advances in *neural architecture search* (NAS) has demonstrated that model architecture has a significant impact on model accuracy and has to be protected [].

Note that, SOTER does not hide information that is revealed by the results of inference queries. This assumption stays in line with the threat model of other previous work, including both cryptographic-based or TEE-based secure inference systems []. Protecting against such attacks that exploit the leakage of inference results is a complementary problem to that solved by SOTER. Indeed, many attacks have been successfully carried out even against systems that completely hide the model with black-box accesses []. However, existing popular mitigations for these attacks, such as differential privacy [], can be combined with SOTER's protocol. We illustrate these attacks and mitigations in more detail in §VII.

SGX's threat model. SOTER has the same threat model for SGX as typical SGX-based systems []. We trust the hardware and firmware of Intel SGX, which ensures that code and data in an enclave cannot be seen or tampered with from outside. However, a client might be corrupted by an adversary, and the adversary can observe and modify model parameters that is partitioned outside enclave. We trust that the remote attestation service can identify genuine SGX devices from fake ones (e.g., emulated with QEMU). Same as DeepEnclave [] and other SGX-based secure inference systems [], we do not intend to address the side-channel vulnerabilities against Intel SGX in this paper.

C. System overview

SOTER is a secure inference system running at the client side. Different from a cloud-side system (e.g., Delphi [31]) that aims to protect the client's sensitive input data, in SOTER, the client's data is never transmitted to the untrusted cloud, hence SOTER naturally protects the client's data. SOTER focuses on protecting the offloaded model at the client side. A client runs SOTER's protocols in a trusted module (i.e., the SGX enclave) that will only fail by crashing, and all system components outside enclave can behave arbitrarily.

SOTER's architecture. Figure 3 shows the architecture of a SOTER client. The client is equipped with an attested

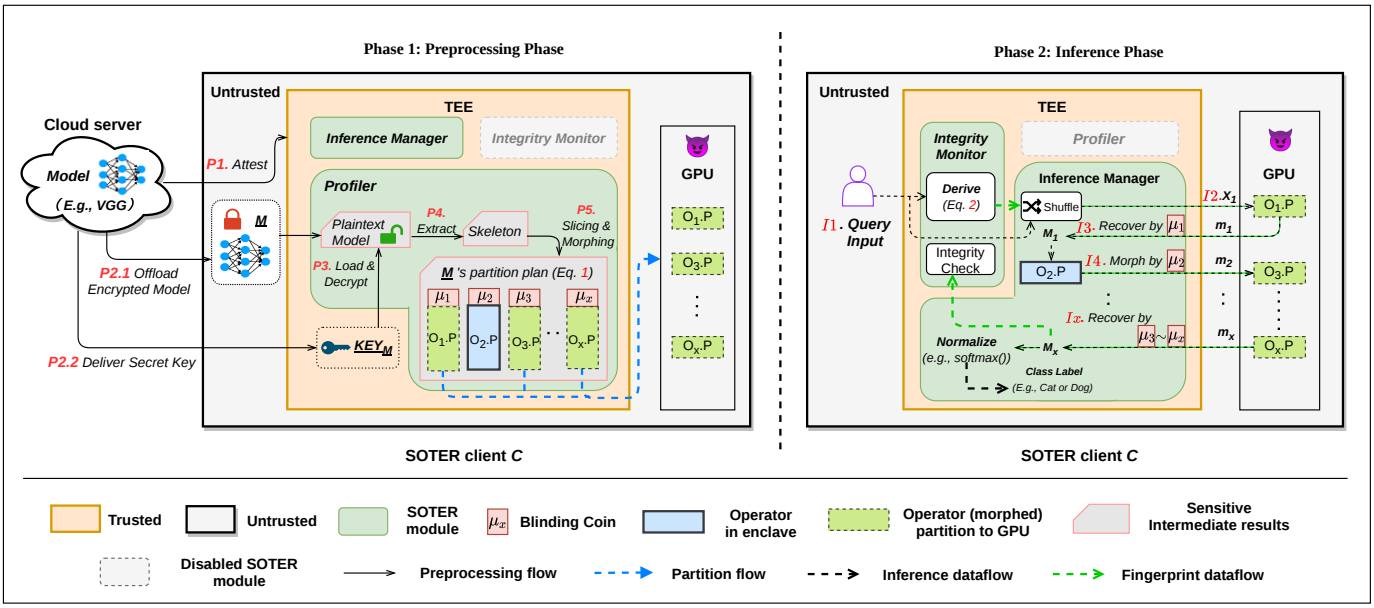


Fig. 3: The architecture of SOTER in a modular manner. SOTER's secure inference protocol has three modules running in the enclave (shielded in green). An adversary can observe and modify partitioned model operators in the co-located GPU.

SGX enclave running only SOTER's inference protocols. Each SOTER client has three trusted modules (in green) running in SOTER's SGX enclave (in orange). The *profiler module* takes an offloaded encrypted model and its corresponding keys as input to extract the model skeleton (i.e., architecture), and runs the *MTR* protocol to find an optimal partition plan. The *inference manager module* takes the client's query input (i.e., data for inference) and manages inference computations with reference to the partition plan. The *integrity monitor module* runs the *oblivious fingerprinting* protocol to derive integrity-checking data flows that are indistinguishable to the client's inference data flows from an adversary's view outside enclave.

SOTER's two-phase design. At a high level, SOTER's protocol for secure inference consists of two phases: an offline *preprocessing phase*, and an online *inference phase*. The offline preprocessing phase is independent of the client's query input which changes regularly. We assume the offloaded model from the server is static, if the model changes, then both parties should re-run SOTER's preprocessing phase. After preprocessing, during the online inference phase, the client sends query input to get the eventual inference result by passing through SOTER's integrity monitor module and inference manager module. Note that, SOTER's two-phase design is most suitable for applications which could reserve sufficient preprocessing resources (see **Remark I**).

SOTER's workflow. Figure 3 shows how SOTER gives black-box protections to a partitioned model without compromising model accuracy, and the inference workflow in SOTER. Without losing generality, we use an image classification model (e.g., VGG-19) as an example to show the workflow.

In the preprocessing phase: (**P1**) The cloud server S attests to the SOTER client C to ensure legitimate SOTER's code is running on top of a genuine SGX-enabled hardware. (**P2.1 & P2.2**) If the attestation fails, then the preprocessing phase aborts; Otherwise, after a successful attestation, S builds up a secure communication channel with C, then S offloads its encrypted model M to the untrusted memory outside enclave because the size of a pre-trained model usually exceeds the memory limit of an enclave (~93.5MB), and S also offloads the corresponding decryption keys KEY_M to C's enclave. (**P3**) SOTER's trusted profiler module loads the encrypted model M in a layer-wise manner and decrypts M with KEY_M. (**P4**) With a plaintext model, SOTER extracts the model skeleton to capture the architecture information including the operator types, the number of layers, and the linkages between layers. (**P5**) With the architecture information, SOTER statically filters out all *associative* operators that meets Equation 1, and then invokes SOTER's *MTR* protocol to securely partition part of model operators to GPU for faster processing.

Specifically, in **P5**, SOTER's *MTR* protocol runs as follows: With a given security parameter θ, SOTER randomly selects $\theta\%$ associative operators, and runs SGX's pseudo-random number generator (PRNG) to produce (1) *fingerprints* of the selected operators for integrity checking (used in the inference phase), and (2) random positive unsigned integers for all operators. These numbers serve as the *blinding coins* to hide the operators' parameters and are always kept in enclave. SOTER morphs every selected associative operator by multiplying each element in the operator's parameter matrices with a blinding coin, and then partitions the selected operators with morphed parameters to GPU. In Figure 3, the selected associative operators (in green) are morphed with corresponding coins, e.g., the parameter of the third operator $O_3.P$ is morphed with a coin μ_3 and partitioned to GPU, while the second operator (in blue) is kept within enclave. In §IV-B, we illustrate our choice of θ by considering the *transferability* of a partitioned model, which is a widely-adopted security metric to quantitatively evaluate the confidentiality of a black-box model [].

In the inference phase, there are two types of dataflows: the inference dataflow (black dotted line) and the fingerprint

dataflow (green dotted line). The life-cycle for the inference dataflow is: (**I1**) The client C sends a query input (i.e., data for inference) to the trusted inference manager module and shuffle with the fingerprint input (introduced below). (**I2**) The inference manager forwards C's input X_1 to the GPU, and the first *morphed* operator $O_1.P$ takes X_1 as input and computes the output m_1. (**I3**) Since the next operator $O_2.P$ is expected to run within enclave according to the plan made in the preprocessing phase, the inference manager takes the computation result of previous partitioned operators (i.e., m_1) as input, *recovers* m_1 with all previous partitioned operators' blinding coins since the last kernel switch (in Figure 3's circumstance, SOTER recovers with only $O_1.P$'s blinding coin μ_1), and gets the recovered result M_1. (**I4**) After that, the second operator $O_2.P$ computes with M_1 and gets an intermediate result (IR), then morphs IR with the $O_2.P$'s blinding coin μ_2 and forwards the result m_2 to the partitioned operator $O_3.P$ in GPU.

The above procedures (**I2**∼**I4**) repeat until all partitioned operators in GPU are computed. In (**Ix**), with the last computation result m_x from $O_x.P$, SOTER recovers the real computation result by using all previous partitioned operators' blinding coins since the last kernel switch (same as the recover step in **I3**). In this case, SOTER recovers with $\mu_2 \sim \mu_x$ and gets the real inference result M_x. Then, with all things done, SOTER runs a standard DNN normalization (e.g., softmax()) to get the class label and return the label to C.

The fingerprint dataflow is used to detect integrity breaches of partitioned operators outside enclave. Specifically, with a set of partitioned operators' fingerprints produced in the preprocessing phase, within the inference dataflow, SOTER's trusted integrity monitor module dynamically derives oblivious fingerprints by utilizing the associativity of Equation 2, injects these fingerprints into the inference dataflow, in order to obviously check whether the parameters of partitioned operators have been modified by the adversary outside enclave. We prove in §IV-D that the adversary outside enclave cannot distinguish the fingerprint dataflow from the inference dataflow, preventing the adversary from bypassing the integrity checking.

Remark I (SOTER's targeting applications). Same as other secure inference systems that involve two phases (Delphi [31], Gazelle []), SOTER is best suited for applications whose inference is latency-sensitive, but is usually not performed frequently enough to take up all computational resources needed for preprocessing. Examples of such applications include speech-to-text systems like [13] and image classification systems like Google Lens [14].

IV. PROTOCOL DESCRIPTION

This section describes SOTER's protocols that provide black-box inference services at the edge while meeting all three crucial requirements. To ensure the offloaded model at the client can only be queried as a black-box oracle without losing confidentiality (**R1**), SOTER's *MTR* protocol (§IV-A) randomly selects $\theta\%$ associative operators (with Equation 1), and morphs these operators' parameters with hidden *blinding coins* in enclave. These morphed operators are then partitioned to the untrusted GPU for faster inference computing (**R2**). Benefiting from the associativity property, SOTER can recover the real inference results in enclave without an accuracy drop (**R3**). We quantitatively analyze the confidentiality guarantee of the *MTR* protocol, especially for discussing the selection of the security-sensitive parameter θ (§IV-B).

TABLE I: SOTER's protocol variables.

Variables	Descriptions
$\Theta(default = 0.8)$	Portion of associative operators partitioned to GPU.
O_g	The set of partitioned operators in GPU.
O_e	The set of operators maintained in enclave.
$O(O_g \cup O_e)$	The whole set of model M's operators.
$Para(O_i)$	Parameters of operator O_i.
FP_{O_i}	Fingeprint of operator O_i.
μ_i	Blinding coin of operator O_i.
$K(default = 10)$	The initial number of fingerprints for each $O_i \in O_g$.
$W(default = 2)$	Top-W operators are always maintained in enclave.

However, using the *MTR* protocol requires partitioning a fraction of operators to GPU, which exposes an attack surface to a corrupted client or malicious OS (the adversary) outside enclave. The adversary can observe and modify the partitioned operators' parameters and perturb the inference results. Motivated by this security challenge, SOTER uses a set of *oblivious fingerprints* that holds the ground-truth inference result of each partitioned operator to detect integrity breaches outside enclave (§IV-C), while ensuring the fingerprint dataflow share the same distribution with the client's inference dataflow, such that the adversary cannot distinguish the fingerprints and bypass our detection (§IV-D).

Similar to existing secure inference systems at the cloud side [31], [22], we now give the formal definition of a partition-based secure inference system at the edge as follows.

Definition I. *A system ψ running at the client C holds an offloaded model M with architecture α and parameters $(M_1, ..., M_l)$, and the client C holds input queries to run local inference on M. ψ is a secure inference system at the edge if it satisfies the following guarantees:*

- **Security (R1):**
  - **Confidentiality.** *We require that a corrupted client C does not learn anything about either the model M's architecture α or the parameters $(M_1, ..., M_l)$. Formally, with a partitioned model M_p under the view of C, we require a low transferability of ψ, which is defined as the ratio of the correctly computed inference examples that the partitioned model M_p can mimic the original model M (more details in §IV-B).*
  - **Integrity.** *Given that a corrupted client C can observe and modify the parameters of partitioned operators, we require that ψ has to detect partitioned operators' integrity breaches with high accuracy and zero false-positives (see analysis in §IV-D).*
- **Performance (R2).** *We require that conducting inference on the offloaded model M should be accelerated with a co-located accelerator (e.g., GPU) in order to meet the stringent inference latency requirements of modern applications at the edge.*
- **Accuracy (R3).** *Given an partitioned model M, on every input query X from client C, we require that the output returned by ψ should achieve exact the same inference result as $M(X)$, i.e., without any accuracy drops.*

A. Morph Then Recover (MTR) protocol

SOTER's *MTR* protocol is divided into two stages: a *morphing* stage in the preprocessing phase (handled by the profiler module), and a *recover* stage in the inference phase (handled by the inference manager module). In the morphing stage, SOTER first makes *blinding coins* for every operators (including both associative and non-associative operators), and then make a partition plan with a given security parameter θ. Then, in the recover stage, SOTER runs inference across GPU and enclave, and recovers inference results in enclave when needed. Algorithm 1 and Algorithm 2 show the *MTR* protocol.

Stage 1-1: losslessly morph with blinding coins. SOTER assigns every operator with a randomly generated unsigned integer, which serves as the blinding coin (in short, coin) to guarantee model confidentiality. In our implementation, we use the trustworthy random source (`sgx_read_rand`) in SGX to generate coins and and assign coins to every operator in model M (Line 3∼8 in Algorithm 1).

SOTER's blinding coin design protects model confidentiality from two aspects. First, if an operator O_i is associative and is selected to be partitioned to GPU according to the profiling plan, then SOTER morphs $Para(O_i)$ by multiplying each element in $Para(O_i)$ with the corresponding coin μ_i. By hiding μ within enclave, SOTER ensures that an adversary outside enclave cannot learn the real parameters before being morphed in enclave.

Second, even if an operator O_i is kept within enclave during inference, we still need its coin μ_i to help hide the coin of *previous* partitioned operators. Consider a case in Figure 3 where O_1 (in GPU) is a linear operator and O_2 (in enclave) is a non-linear `ReLU` operator. If we only morph O_1 and do not assign a coin to O_2, then, with a client input X, O_1 outputs $y_1 = X(\mu_1 * O_1.P)$ and O_2 outputs $y_2 = ReLU(y_1/\mu_1) = ReLU(X * O_1.P)$. Since the `ReLU` operator just filters out negative values in parameter matrices and do not transform non-zero values, an adversary who observes both y_1 and y_2 will directly infer the value of coin μ_1, violating the confidentiality of the partitioned O_1 (concretely, $Para(O_i)$). However, since SOTER assigns coin μ_2 to O_2, O_2 outputs $y_2 = \mu_2 * ReLU(X * O_1.P)$, an adversary can observe only the value of $\mu_1\mu_2$ but cannot infer μ_1 or μ_2 respectively.

Remark II (full precision arithmetic). Using coins to hide parameters of partitioned operators requires multiplication of a parameter matrix by a secret number. As popular implementations of neural network inference perform arithmetic over 64-bit floating-point numbers [], in order to morph parameters without losing precision, similar to Delphi [31] and Slalom [38], in our implementation we choose to generate 16-bit unsigned integers as coins which prevent multiplication values from growing exponentially with the number of multiplications (and thus overflowing).

Stage 1-2: automatically partition associative operators. With a given parameter θ, SOTER automatically partitions a portion of associative operators (that are morphed) to GPU for inference acceleration. The function *partition()* in line 9∼22 of Algorithm 1 shows the pseudo code.

Specifically, in the preprocessing phase, SOTER iterates every operator in model M and randomly selects a portion

Algorithm 1: MTR protocol at client c (offline) (§IV-A).

```
▽ Preprocessing Phase (within enclave).
1  Initialization
2      μ ← ∅;
3  Function make_coin() do
4      foreach operator op in O do
5          index ← O.index(op);
6          coin ← ∅;
7          while coin = ∅||coin = sgx_read_rand() < 0 do
8              μ ← coin;
9  Function partition() do
10     foreach operator op in O do
11         op.partition ← False;
12         op.index ← O.index(op);
13         if op.index < W then
14             if 0 < normalize(sgx_read_rand()) < Θ then
15                 op.partition ← True;
16             if op is associative & op.partition then
17                 foreach element e in Para(op) do
18                     e ← μ_index × e ;        // Morph.
19                 O_g.add(op) ;      // GPU operators.
20             else
21                 O_e.add(op) ;      // TEE operators.
22     O_g.copyTo ("cuda") ;     // Partition to GPU.
```

of operators and add to the partition set O_g. When meets an associate operator, SOTER uses the trustworthy source of SGX to generate a randomly number for the operator, and then normalize the number. If the normalized number is within the range of $0 \sim \theta$, then the operator is selected to join O_g. Concretely, with $\theta = 0.8$, an associative operator would have an 80% chance to be partitioned to GPU. Then, after all operators are iterated, SOTER partitions all operators in O_g to GPU for inference acceleration. Note that, SOTER will always hold top-W (W=2 by default) operators in enclave even if these operators are associative (Line 13). This design choice is made to ensure the input to the first partitioned operator in GPU (i.e., X_1 in Figure 3) is always oblivious, such that SOTER can obliviously check the integrity of partitioned operators by injecting fingerprint dataflows (see more details in §IV-C).

We now give examples of running SOTER's *MTR* protocol to partition neural networks (shown in Figure 4). We use two DNN models that achieve state-of-art performance on typical deep learning tasks: a VGG-19 model for computer vision task and a WaveRNN model for natural language processing task. VGG-19 is a popular CNN model known for its high-inference accuracy for image classification tasks []; WaveRNN is a state-of-the-art RNN model developed by Deepmind and Google for efficient speech-to-text transformations []. To illustrate our protocol in a lucid manner, in this example we choose $\theta = 1$, which means that all associative operators will be partitioned to GPU for acceleration.

The partition plans for typical layers of these two models are shown in Figure 4 (with $\theta = 1$). For VGG-19, SOTER can partition most of its operators to GPU because (1) VGG-19 is a typical layered model composed by associative operators including `convolution`, `ReLU` and `max-pool`, recalling

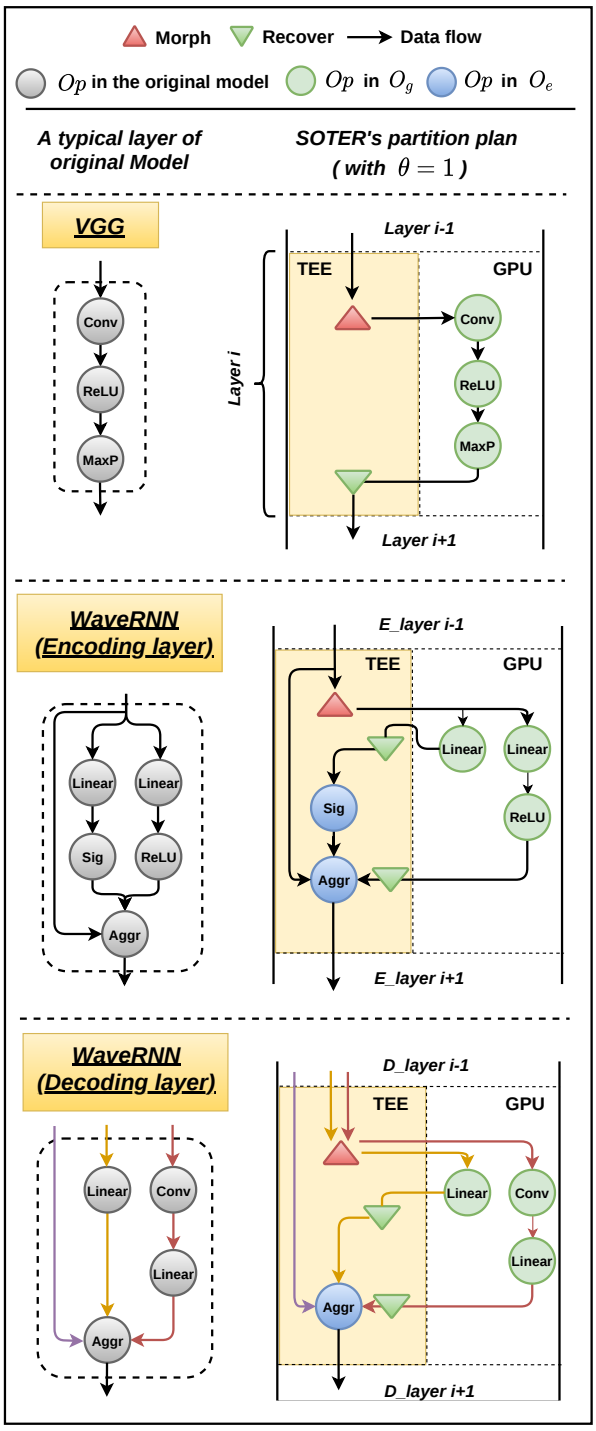


Fig. 4: SOTER's partition plan on two typical DNN models.

that the two non-linear operators can also meet the associativity property given a positive coin (§II-A). (2) Most non-associative operators in VGG-19 such as `BatchNormal2d` and `Dropout` are de-activated throughout the inference process to avoid non-determinism []. For WaveRNN, we give partition plans for both its encoding layer and decoding layer because WaveRNN is built by iteratively stacking these two layers. We see that different from VGG-19, WaveRNN has more non-associative operators (e.g., `Sigmoid`) that cannot be partitioned to GPU. This indicates that SOTER would achieve a more significant inference speedup when runs on convolution networks, which is proved in our evaluation §V.

Algorithm 2: MTR protocol at client c (online) (§IV-A).

```
▽ Inference Phase (cross enclave & GPU).
1  Function secure_inference do
       ▽ I1.
2      every client input X do
3          X.copyTo ("enclave");
4          foreach index i ∈ range(W) do
5              X ← O_i(X) ; // Top-W TEE Inference.
6          start ← find the next op in O that belongs to O_g;
7          end ← find the next op in O that belongs to O_e;
8          while start < |O| do
               ▽ I2.
9              X.copyTo ("cuda");
10             foreach index i ∈ range(start, end) do
11                 X ← O_i(X) ;      // GPU Inference.
               ▽ I3.
12             X.copyTo ("enclave");
13             X. ← ∏_{start}^{end-1} μ_i^{-1} × X ;      // Recover.
14             start ← end;
15             end ← start from O_end find the next op in O
                   that belongs to O_g;
16             foreach index i ∈ range(start, end − 1) do
17                 X ← O_i(X) ;      // TEE Inference.
               ▽ I4.
18             X ← ∏_{start}^{end-1} μ_i × X ;      // Morph.
19         X ← normalize(X);
20         return Top_1(X) ;      // Inference result.
```

Stage 2: black-boxed online inference. Now we present the end-to-end black-box inference process given that we have prepared coins and a portion of associative operators have been partitioned to GPU. As shown in Algorithm 2, SOTER first computes the inference result for the top-W operators maintained in enclave. Then, SOTER iterates **I2**∼**I4** as follows. **I2**: SOTER finds the longest length of consecutively partitioned operators in O_g and computes the inference result by forwarding the input through these operators (line 6∼11). **I3**: Then SOTER copies the result back to enclave, recovers the real inference result by multiplying the reciprocal coins of all operators in last step (line 13), and then forwards the result to operators maintained in enclave (before the next copy to GPU). **I4**: SOTER morphs the forwarding result with all coins of enclave operators in last step (line 18), and then copies back to GPU. This procedure terminates when all partitioned operators in GPU are iterated. Then SOTER normalizes the final result and returns the result to the client.

B. Security analysis of MTR protocol

Attack assumptions. In SOTER, same as existing two-party inference systems, we assume both parties are curious to learn the private information of the other party: we assume a corrupted client who is curious to learn the model offloaded from the server, and the server also tries to learn the client's private input. Since SOTER restricts data at the edge device

and never uploads data to the cloud, we focus on analyzing how much information a corrupted client (the adversary) can learn in our security analysis. Specifically, we consider the adversary could be a business competitor whose purpose is to steal the offloaded model by using the partition information that he observes to train a *substitute model*, in order to reduce the competitive advantages of model owners.

Recall that we assume the adversary has a bounded computational capability (§III-B), and does not know the whole training dataset for the offloaded model (otherwise it is totally unnecessary for the adversary to "*steal*" the model). This assumption stays in line with existing partition-based inference systems [] to evaluate the *black-box* capability of a model's partition plan.

Evaluation setup. We evaluated the security on *MTR* protocol with three CNN models and one RNN model. For the CNN models, we used VGG-19 [], NASNet [], and PNASNet [], and pre-trained them on the widely used CIFAR-10 dataset []; For the RNN model, we used a WaveRNN model and pre-trained it on the popular GTA dataset []. With the bounded computational capability assumption, we isolated 90% of training data (e.g., 45,000 images in CIFAR-10), and the remaining 10% of training samples are used by the adversary. We assume the adversary can enrich his training dataset to train an *substitute model* by using data augmentation techniques [].

θ-transferability analysis. Recall our definition on secure model inference at the edge (**Definition I**), we now analyze the confidentiality guarantee of SOTER's *MTR* protocol by evaluating the effects that parameter θ could have on the quality of the adversarial substitute model SM. With a given θ, we call a SOTER partitioned model outside enclave as $PM(\theta)$, whose architecture and parameters are completely observable by the adversary. The adversary then uses $PM(\theta)$ as the basic model (with both $PM(\theta)$'s architecture and parameters) to train a substitute SM.

Same as prior work [], we used SM's *θ-transferability* to evaluate the resultant security guarantee of SOTER's MTR protocol with a given partition parameter θ. Transferability means the portion of functionalities that SM can mimic the original offloaded model M. A higher transferability indicates that the SM trained by the adversary can mimic a larger portion of functionalities (i.e., more confidentiality loss), while a lower transferability reveals a smaller portion of functionalities are stolen (i.e., less confidentiality loss). Concretely, for CNN models, we use the *Top-1* accuracy as the transferability metric and we use the *BLEU score* for the RNN model. Both of them are well-known AI metrics to measure the quality of a trained model.

As shown is Figure 5, the red line (VGG-19) can achieve the ground-truth Top-1 accuracy (74.5%) when the X-axis is "*White*", i.e., a scenario where we placed the entire VGG-19 in GPU without activating SOTER's protocol. The ground-truth accuracy result is intuitive due to the white-box model being the same as the original model M. Next, we activated SOTER's *MTR* protocol to evaluate the effects that θ brings to Top-1 accuracy (i.e., transferability). When θ downgrades from 1 to 0, we observed a significant accuracy drop from 71.8% to 54.3%. Specifically, we collected three observations from our evaluation. **O1:** When $\theta = 1$, SM's accuracy decreases ~2.7%

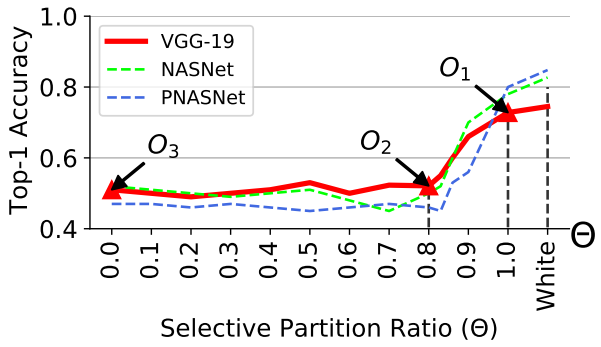


Fig. 5: θ-transferability analysis of three CNN models. For CNN models, we use the Top-1 inference accuracy as the transferability metric.

because in that case the adversary can train SM with the knowledge of: (1) the partial architecture of model M (since operators used in VGG-19 are mostly associate, the adversary will get most of the architecture information when $\theta = 1$), and (2) morphed parameters of all associative operators partitioned to GPU. With these knowledge, the adversary trains SM with his limited training dataset and finally achieve a resultant substitute model SM with a relative high accuracy. **O2:** But when SOTER chooses $\theta = 0.8$, we observed a significant accuracy drop to 54.7%. This is because, SOTER only partitions an associate operator to GPU with 0.8 probability. Hence, we hide the architecture information of the model, and the adversary can only train SM with limited architecture information and morphed parameters of only a fraction of partitioned associate operators. As a result, the adversary can only train a low-accuracy model SM which cannot substitute the original high-accuracy model M at all. **O3:** When θ is within the range from 0 to 0.8, we observed that the accuracy is almost steady without spikes. This is because, even with a smaller θ which means less associative operators are partitioned to GPU, the difficulty for the adversary to train a substitute model SM remains almost unchanged. Even when $\theta = 0$, which means that no operators are partitioned to GPU (i.e., fully black-boxed), the adversary can still train from a blank model by using the adversarys training dataset.

We also conducted the transferability analysis upon NASNet, PNASNet and WaveRNN, shown in Figure 5 and Figure 6. The similar downgrading trend and *θ-transferability* relationship reveal that hiding around 80% architecture information could be enough to guarantee sufficient model confidentiality. Hence, we choose $\theta = 0.8$ in SOTER by default, which obtains the maximum performance benefit (most associate operators would be partitioned to GPU for fast processing) when achieving comparable security level as fully black-boxed models (i.e., $\theta = 0$).

Comparison to baselines. Compare to SOTER, existing TEE-based inference systems at the edge including MLcapsule [19], eNNclave [34] can also achieve a low transferability. This is because, MLcapsule runs the entire model within SGX at the cost of higher inference latency, and eNNclave partitions manually-tuned model parameters to GPU at the cost of inference accuracy. Benefiting from SOTER's *MTR* protocol, SOTER is the first work that achieves low transferability, low latency and high accuracy simultaneously.

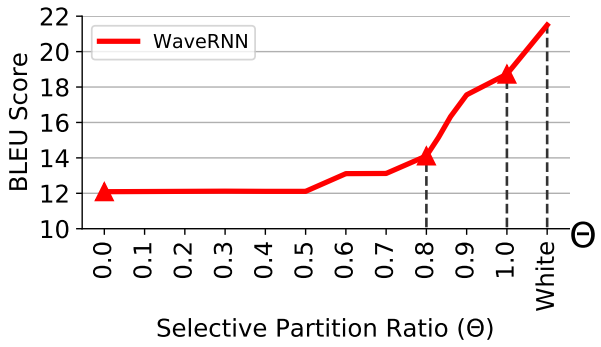


Fig. 6: θ-transferability analysis of one typical RNN model WaveRNN. We use the BLEU inference score as the transferability metric.

Algorithm 3: Fingerprint preparing (offline) (§IV-C).

```
▽ Preprocessing Phase (within enclave).
Function preparing() do
    foreach k in range(0, K) do
        input ← generate_random_input(M);
        FP_0[k].input ← input;
        FP_0[k].output ← O_0(input);
        foreach operator O_i in O_g[1, |O_g|] do
            FP_i[k] ← ⟨input : FP_{i-1}[k].output, output :
              O_i(FP_{i-1}[k].output)⟩
```

C. Detecting integrity breaches with oblivious fingerprints

In order to detect integrity breaches of partitioned operators outside enclave, SOTER's starting point is to use a simple but effective re-compute technique. The main concept of this idea is to generate checking queries in enclave at the runtime, inject these checking queries to the normal inference dataflow to go through the entire computing process of partitioned operators, and compare the returned inference results of checking queries with expected outputs in enclave. SOTER will report a suspicious integrity breach to the client if the inference result differs from the expected output.

Unfortunately, simply using the re-compute technique may cause a severe security problem: the adversary can persistently monitor the inference process and observe every intermediate result (IR) transferred from enclave to GPU. If using only a static set of checking queries, the adversary can easily distinguish those fixed checking queries among all IRs, hence bypassing SOTER's integrity check by computing and returning the correct inference result to the enclave. However, if we trivially use different checking queries for every integrity check (such that the adversary cannot determine the fixed checking queries), then we have to generate random checking query input and compute an operator's expected output at enclave before we send the checking query to the partitioned operator in GPU, which is extremely costly as we have to run slow CPU inference computing in enclave for each partitioned operator and for each client's inference input.

SOTER tackles this challenge by making all checking queries indistinguishable (oblivious) from other normal inference queries, and efficiently generating those oblivious checking queries without needing to compute the expected checking outputs in enclave. Specifically, SOTER's oblivious fingerprinting technique can be divided into two steps. First, SOTER makes a set of *cornerstone fingerprints* for each partitioned operators during the preprocessing phase, which serve as the basis of checking queries. Second, during the inference phase, SOTER dynamically derives oblivious fingerprints from existing cornerstone fingerprints prepared in the preprocessing phase. By utilizing the variant of the associativity probability (Equation 2), SOTER can skip the step of computing the expected checking outputs in enclave before sending these checking queries to GPU. The details are as follows.

Step 1: preparing cornerstone fingerprints. SOTER prepares cornerstone fingerprints after a partition plan is made and before a fraction of operators are partitioned to GPU (i.e., between line 21∼22 of Algorithm 1). Algorithm 3 shows the preparing procedure. SOTER prepares K (by default K=10) cornerstone fingerprints for each partitioned operator during the processing phase. A fingerprint is a two-element tuple: $\{input, output\}$, where the input is randomly generated in enclave (line 3) and the output is computed by forwarding the input through a ready-to-partition operator (line 5). The whole preparing procedure is running within enclave. By doing so, we ensure the correctness of each fingerprint output, Now we hold K correct input-output tuples (i.e., fingerprints) for each partitioned operator.

Step 2: dynamically derive oblivious fingerprints. SOTER dynamically derives oblivious fingerprints at runtime without needing to compute expected outputs for each partitioned operator. Specifically, with a fixed set of cornerstone fingerprints $FP_O[K]$ for each partitioned operator O prepared at **Step 1**, SOTER randomly selects T fingerprints and generates T random numbers for each fingerprint by using SGX's trustworthy random number generator. Then, SOTER applies the associativity property (i.e., Equation 2) to generate a new fingerprint FP_{New} by using the T cornerstone fingerprints and random numbers. Concretely, the input for FP_{New} is $\sum_{i=1}^{T} T_i * FP_O[i].input$, i.e., we multiply each selected cornerstone fingerprint's input with a corresponding number and adds them all as the new input; For the output, according to Equation 2, the associativity property implies that the corresponding output for a group of associated operators has the same transformation as the input, thus we can directly get the expected output for FP_{New} as $\sum_{i=1}^{T} T_i * FP_O[i].output$, without needing to conduct one costly CPU inference for the given operator O.

D. Proof of fingerprints' obliviousness

V. EVALUATION

Testbed. We conducted all evaluation in our cluster on a GPU farm of 10 DELL R430 servers with 20 Nvidia RTX2080TI graphic cards. Each graphic card is connected to the host with PCIe 3.0. Each server is equipped with an Intel 2.60GHz E5-2690 V3 CPU, 64GB memory, and 40Gbps NIC. We ran SGX with the real Intel SGX hardware in our cluster.

Model description. As shown in Table II, our evaluation covers four typical types of DNN models. All models can achieve state-of-the-art inference accuracy on domain-specific tasks with well-trained architectures and parameters. These models

Task	Pattern	Model	Dataset	Defaulte Input Tensor Shape
Image Classification	MLP	MNIST-MLP [27]	MNIST [26]	(1, 1, 28, 28)
	CNN	AlexNet [25]	CIFAR-10 [24] and ImageNet [8]	(1, 3, 224, 224)
		DenseNet-121 [21]		
		VGG-19 [36]		
		ResNet-101 [20]		
		NASNet [44]		(1, 3, 331, 331)
		PNASNet [30]		
Translation	RNN	WaveRNN [23]	WMT16 (En-De) [1]	512 tokens
	Transformer	Nvidia-Transformer []	WMT16 (En-De) [1]	512 tokens

TABLE II: Overview of our evaluation tasks.

include: a Multi-layer Perception (MLP) model, six Convolution Neural Network (CNN) models including AlexNet [25], DenseNet-121 [21], VGG-19 [36], ResNet-101 [20], NASNet [44] and PNASNet [30], a Recurrent Neural Network (RNN) model WaveRNN [23] and a transformer model [].

Among these models, the MLP model is purely composed by the same linear operators (`torch.nn.Linear`) and ReLU activation functions (`torch.nn.ReLU`); For the CNN models, AlexNet, DenseNet-121, VGG-19, and ResNet-101 are popular models that achieve high inference accuracy on computer vision tasks. In addition to these CNN models, we evaluated two large CNN models NASNet and PNASNet which are designed by recent *neural architecture search* (NAS) techniques. Each of the NAS searched model has around 3GB parameters; WaveRNN is

Datasets description. The parameters of the models are trained by us using well-known datasets on PyTorch. We use CIFAR-10 [24] and ImageNet [8] for CV models and WMT16 [] for NLP models.

Baselines. We compared SOTER with MLCapsule [19], an SGX-based secure inference system at the client side. MLCapsule shields the entire model in SGX and executes inference tasks only with CPU. Compare with MLCapsule, SOTER securely outsources part of the computation from SGX to the GPU. It is important to note that SOTER is orthogonal with the secure inference systems at the server-side such as Slalom [38]. This is because they are designed to protect data from the user while SOTER is designed to protect the model from the model provider (e.g., Google [42]).

We also compared SOTER with two state-of-the-art partition-based secure inference systems (SOTER is also partition-based): eNNclave [34] and DeepEnclave [17]. eNNclave is a notable open-sourced inference system. eNNclave puts last several task-specific dense layers in SGX, others in GPU. SOTER has the same confidentiality guarantee as eNNclave. DeepEnclave is not open-sourced, so we built a proof-of-concept implementation, which can achieve the same performance as reported in their paper. In contrast to eNNclave, DeepEnclave puts the first several layers in SGX, because these layers could reveal knowledge about user input data. Since SOTER's client is never exposed to a third party, SOTER has the same confidentiality guarantee on user input data. Both of these two systems need manual partition with expert knowledge, and they need to transfer outsourced (to GPU) layers' parameters (drop accuracy).

We divide our evaluation into two sections that answer the following questions:

§V-A How efficient is SOTER compared with existing secure inference systems?

§V-B How accurate is SOTER compared with existing secure inference systems?

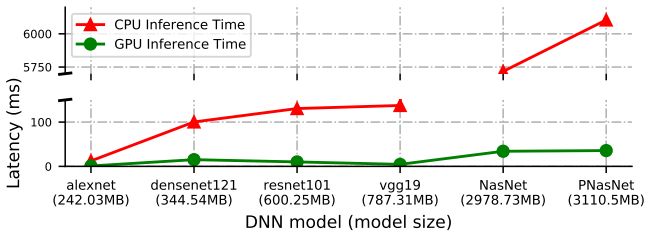


Fig. 7: The huge latency gap between CPU and GPU inference. We repeat each model inference 10,000 times and report the average latency.

A. *End-to-end Performance*

End-to-end inference latency. We first investigated the end-to-end inference latency of SOTER and three baseline secure inference systems with eight models. All reported measurements are the average inference latency of 20 independent inferences on the same input. As shown in Figure **??**, the partition-based secure inference systems including SOTER (execute inference tasks with both CPU and GPU) achieved better performance than MLCapsule (executes inference tasks only with CPU). This is because the CPU is at least one order of magnitude less efficient than GPU [38] in executing the inference tasks.

Among all baseline systems, SOTER achieved the lowest inference latency for all models. Moreover, as the model size increased, the latency gaps between SOTER and the baseline systems grew larger. For instance, with a small DNN model (AlexNet, 242.03MB), SOTER achieved 21.5% lower inference latency on average than existing secure inference systems. With a large DNN model (PNasNet, 3110.5MB), SOTER achieved 85.3% lower inference latency on average than existing secure inference systems.

Latency breakdown. To understand SOTER's low inference latency, we broke down the time taken in different phases of an inference task in SOTER, and the results are reported in Table III. It can be clearly observed that, for all models, a large majority of the inference latency in SOTER was spent on GPU. This is because SOTER securely offloads most computational-expensive model layers to GPU, and computes very few lightweight model layers in SGX, resulting in low end-to-end inference latency. Doing so requires SOTER to copy the model layer parameters from SGX to the GPU memory only in the preprocessing phase. During the inference phase, SOTER only copies query inputs (much smaller than the model parameters) into the GPU memory, and copies the outputs back to the SGX enclave memory for each layer. Frequent memory copies between GPU and SGX may cause a substantial latency penalty. To amortize these memory copy overhead, SOTER batches several continuous layers together, and performs input/output memory copies only before and after computing the batch of layers.

Overall, SOTER can achieve a low inference latency, and the computation overhead (CPU computation in SGX and kernel switch) caused by enclaving the sensitive layers is moderate and increases sub-linearly with the model size.

SGX to GPU communication cost. We also measured the *parameter size* and the *forward pass size* for a single inference task of different models in SOTER, where *parameter size* is the total size of all parameters of the model, and the *forward pass size* indicates the sum of the *output* size from each layer in the model. Table **??** summarizes the results. As can be observed from Table III and Table **??**, the inference latency of the models shows a similar trend as their forward pass sizes. When the forward pass size becomes larger, there is more intermediate input and output data be transferred among layers. This requires SOTER to perform more memory copy of these input and output data among GPU and SGX.

Models	Preprocessing (ms)	Inference (ms)		
		SGX (CPU)	Kernel Switch	GPU
MLP		0.01	0.01	0.02
AlexNet		2.475	1.23	4.03
VGG-19		2.59	1.29	5.03
ResNet-101		2.30	1.65	13.10
DenseNet121		3.13	2.56	25.77
NASNet		4.43	3.21	37.71
PNASNet		4.63	3.31	39.41
WaveRNN		12350.25	6175.13	104975.35
Transformer				

TABLE III: End-to-end latency breakdown of SOTER.

B. Inference Accuracy

In this section, we evaluate the resulting inference accuracy of SOTER and existing secure inference systems.

For each system, we compared the inference accuracy of the protected models (i.e., running some or all layers of the ML model in SGX enclaves to protect the model) and the corresponding white-box unprotected model (i.e., running the entire model on GPU without the protection of SGX).

As shown in Table **??**, SOTER and MLCapsule achieved zero accuracy drop, while other partition-based systems caused inference accuracy drops from 2% to 5%. MLCapsule was able to achieve zero accuracy loss because it shields the entire model in the SGX enclave to protect the highest security guarantee. However, at the same time, the security guarantee of MLCapsule come at a steep price in performance, compared to untrusted alternatives (i.e., perform inference tasks on the GPU).

To achieve better performance, the partition-based secure inference systems outsource some inference computations to the untrusted GPU. However, existing partition-based systems outsource some layers to the GPU by retraining these outsourced layers, which changes the parameters of the outsourced layers, leading to a drop in the inference accuracy.

By safely morphing and outsourcing computational-expensive layers to GPU for faster inference computing, and then recovers the real inference result in SGX, SOTER achieves the same model sensitivity (§IV-B) and integrity (§IV-D) guarantee with other partition-based systems without sacrificing the inference accuracy.

VI. DISCUSSION

VII. RELATED WORK

A. Secure Inference Protocols

In this section, we discuss prior work about secure inference over neural networks. There are two categories of secure inference protocols: client-side protocols and server-side protocols. Client-side protocols are designed to protect the models provided by the model providers. A client-side protocol runs on edge clients and the clients submit queries to the local model protected by the protocol. Server-side protocols are designed to protect the client's query inputs. A server-side protocol runs on the servers (e.g., clouds [13]), and the clients upload their queries to the server.

TEE-based protocols. Slalom [38] is a cloud-side secure inference protocol. Slalom partitions the DNN computations between the trusted TEE and the untrusted but high-performance GPU. Like SOTER, Slalom also splits the inference into the offline (preparing SOTER) and online (inference in SOTER) phases, and compute both phases in TEE. Unlike SOTER, Slalom is server-side, and is designed to protect the query input, while SOTER is designed to protect the model.

VIII. CONCLUSION

REFERENCES

[1] "Wmt16," http://www.statmt.org/wmt16/translation-task.html, 2016.

[2] A. Aggarwal, T. E. Carlson, R. Shokri, and S. Tople, "Soteria: In search of efficient neural networks for private inference," *arXiv preprint arXiv:2007.12934*, 2020.

[3] J. Aumasson and L. Merino, "Sgx secure enclaves in practice–security and crypto review," *Black Hat*, 2016.

[4] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[5] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," *arXiv preprint arXiv:1812.00332*, 2018.

[6] V. Costan and S. Devadas, "Intel sgx explained." *IACR Cryptol. ePrint Arch.*, vol. 2016, no. 86, pp. 1–118, 2016.

[7] P. Das, N. Ivkin, T. Bansal, L. Rouesnel, P. Gautier, Z. Karnin, L. Dirac, L. Ramakrishnan, A. Perunicic, I. Shcherbatyi *et al.*, "Amazon sagemaker autopilot: a white box automl solution at scale," in *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, 2020, pp. 1–7.

[8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.

[9] S. Durga, E. Daniel, and S. Deepakanmani, "Dnn-based decision support system for ecg abnormalities," in *2nd EAI International Conference on Big Data Innovation for Sustainable Cognitive Computing*. Springer, 2021, pp. 331–338.

[10] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *The Journal of Machine Learning Research*, vol. 20, no. 1, pp. 1997–2017, 2019.

[11] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *International Conference on Machine Learning*. PMLR, 2016, pp. 201–210.

[12] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[13] Google, "Cloud speech-to-text api -language support," https://cloud.google.com/speech-to-text/docs/languages.

[14] ——, "Google lens," https://lens.google/.

[15] T. Graepel, K. Lauter, and M. Naehrig, "Ml confidential: Machine learning on encrypted data," in *International Conference on Information Security and Cryptology*. Springer, 2012, pp. 1–21.

[16] L. Greco, P. Ritrovato, and F. Xhafa, "An edge-stream computing infrastructure for real-time analysis of wearable sensors data," *Future Generation Computer Systems*, vol. 93, pp. 515–528, 2019.

[17] Z. Gu, H. Huang, J. Zhang, D. Su, A. Lamba, D. Pendarakis, and I. Molloy, "Securing input data of deep learning inference systems via partitioned enclave execution," *arXiv preprint arXiv:1807.00969*, 2018.

[18] P. Guo, B. Hu, and W. Hu, "Mistify: Automating dnn model porting for on-device inference at the edge." in *NSDI*, 2021, pp. 705–719.

[19] L. Hanzlik, Y. Zhang, K. Grosse, A. Salem, M. Augustin, M. Backes, and M. Fritz, "Mlcapsule: Guarded offline deployment of machine learning as a service," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 3300–3309.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[21] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.

[22] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A low latency framework for secure neural network inference," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 1651–1669.

[23] N. Kalchbrenner, E. Elsen, K. Simonyan, S. Noury, N. Casagrande, E. Lockhart, F. Stimberg, A. Oord, S. Dieleman, and K. Kavukcuoglu, "Efficient neural audio synthesis," in *International Conference on Machine Learning*. PMLR, 2018, pp. 2410–2419.

[24] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[25] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, pp. 1097–1105, 2012.

[26] Y. LeCun, "The mnist database of handwritten digits," *http://yann. lecun. com/exdb/mnist/*, 1998.

[27] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[28] J. Li, D. Yu, J.-T. Huang, and Y. Gong, "Improving wideband speech recognition using mixed-bandwidth training data in cd-dnn-hmm," in *2012 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2012, pp. 131–136.

[29] Y. Lin, H. Zhao, Y. Tu, S. Mao, and Z. Dou, "Threats of adversarial attacks in dnn-based modulation recognition," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2469–2478.

[30] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, and K. Murphy, "Progressive neural architecture search," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 19–34.

[31] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *29th {USENIX} Security Symposium ({USENIX} Security 20)*, 2020, pp. 2505–2522.

[32] M. H. Mofrad, A. Lee, and S. L. Gray, "Leveraging intel sgx to create a nondisclosure cryptographic library," *arXiv preprint arXiv:1705.04706*, 2017.

[33] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, and L. Benini, "A 64-mw dnn-based visual navigation engine for autonomous nano-drones," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8357–8371, 2019.

[34] A. Schlögl and R. Böhme, "ennclave: Offline inference with model confidentiality," in *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, 2020, pp. 93–104.

[35] M. S. Seyfioglu, B. Erol, S. Z. Gurbuz, and M. G. Amin, "Dnn transfer learning from diversified micro-doppler for motion classification," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 55, no. 5, pp. 2164–2180, 2018.

[36] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[37] M. Su, D. W. Hayati, S. Tseng, J. Chen, and H. Wei, "Smart care using a dnn-based approach for activities of daily living (adl) recognition," *Applied Sciences*, vol. 11, no. 1, p. 10, 2021.

[38] F. Tramer and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," *arXiv preprint arXiv:1806.03287*, 2018.

[39] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction apis," in *25th {USENIX} Security Symposium ({USENIX} Security 16)*, 2016, pp. 601–618.

[40] T. X. Tran, M.-P. Hosseini, and D. Pompili, "Mobile edge computing: Recent efforts and five key research directions," *IEEE COMSOC MMTC Commun.-Frontiers*, 2017.

[41] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.

[42] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey *et al.*, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.

[43] Q. Zhang, C. Xin, and H. Wu, "Gala: Greedy computation for linear algebra in privacy-preserved neural networks," *arXiv preprint arXiv:2105.01827*, 2021.

[44] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8697–8710.

[45] P. Zuo, Y. Hua, L. Liang, X. Xie, X. Hu, and Y. Xie, "Sealing neural network models in secure deep learning accelerators," *arXiv preprint arXiv:2008.03752*, 2020.